Universidad / Facultad (completar)

Carrera de Ingeniería (completar)

Laboratorio 7.2: Identidad Visual con Material You

Color dinámico, tipografía Poppins y componentes Material Design 3 en la aplicación EcoTrip 2026

Desarrollo de Aplicaciones Móviles · RDA-2 · Grupo 4

James Argüello

Wismar Benítez

Carlos Galarza

Mateo Morales

Ángel Loor

Asignatura (completar)

Docente (completar)

Octubre de 2026

Introducción

Una aplicación móvil se juzga, antes que por su lógica, por cómo se ve y cómo se lee. Material Design 3, conocido como Material You, propone que la interfaz no sea un objeto estático sino un sistema que se adapta a la persona: a partir del fondo de pantalla del usuario, Android 12 (API 31) y posteriores extraen una paleta tonal y la ofrecen a las aplicaciones como *color dinámico* (Google, s. f.-a). Este laboratorio aplica esa idea a la aplicación EcoTrip 2026, construida en el examen práctico anterior, con tres cambios: activar y verificar el esquema de color dinámico, sustituir la fuente del sistema por Poppins mediante Google Fonts descargables, y estrenar una pantalla con *TopAppBar* y *FloatingActionButton* que reaccionen a los roles de color del sistema.

El informe documenta cada fase, muestra la evidencia en un emulador con Android 16 (API 36) con dos fondos de pantalla distintos, mide el contraste resultante frente a los umbrales de accesibilidad WCAG y cierra con conclusiones y recomendaciones.

Objetivos

1. Configurar el esquema de color dinámico (*dynamicColorScheme*) para que la aplicación se armonice con el entorno del usuario.

2. Integrar una fuente de Google Fonts en *Type.kt* sin incluir archivos binarios en el APK.

3. Implementar componentes Material 3 (*TopAppBar* y *FloatingActionButton*) que consuman los roles de color del tema.

4. Explicar cómo el *colorScheme* influye en la accesibilidad, con mediciones propias.

Marco teórico

Color dinámico. En Jetpack Compose, *dynamicLightColorScheme* y *dynamicDarkColorScheme* devuelven un *ColorScheme* generado a partir del fondo de pantalla del dispositivo (Android Developers, s. f.-a). El esquema no es un color suelto: es un conjunto de roles emparejados (*primary* con *onPrimary*, *primaryContainer* con *onPrimaryContainer*, etc.) cuyos tonos Material calcula para mantener una relación de luminosidad predecible entre el fondo y su contenido (Google, s. f.-b). Por eso, si el código elige siempre el par de roles correcto, el contraste se conserva aunque la paleta cambie por completo.

Fuentes descargables. La biblioteca *ui-text-google-fonts* permite declarar una *GoogleFont* y un *GoogleFont.Provider* apuntando a Google Play Services, que descarga y cachea la tipografía bajo demanda. El proveedor se autentica con un arreglo de certificados (*font_certs.xml*), lo que evita empaquetar archivos TTF en la aplicación (Android Developers, s. f.-b).

Accesibilidad del color. Las Pautas de Accesibilidad para el Contenido Web (WCAG) 2.2 exigen una relación de contraste mínima de 4,5:1 para texto normal y de 3:1 para texto grande y componentes de interfaz (criterios 1.4.3 y 1.4.11, nivel AA) (World Wide Web Consortium [W3C], 2023). Aunque nacieron para la web, Android las adopta como referencia en sus guías de accesibilidad (Android Developers, s. f.-c).

Desarrollo

Adaptaciones respecto a la guía

La guía está escrita para el proyecto «SafePass 2026». El repositorio del grupo contiene EcoTrip 2026, por lo que se aplicaron los mismos pasos con los nombres reales: el paquete es *com.example.myapplication* y la función de tema es *MyApplicationTheme* (en lugar de *SafePass2026Theme*). El texto de la pantalla de prueba se conservó tal como lo pide la guía.

Fase 1. Dependencias y certificados

Se añadió la biblioteca *androidx.compose.ui:ui-text-google-fonts:1.7.0* al catálogo de versiones (*libs.versions.toml*) y se referenció desde *app/build.gradle.kts*; el proyecto usa el BOM de Compose 2024.09.03, que alinea esta biblioteca con el resto de módulos de Compose. El proyecto no incluía *font_certs.xml*, de modo que se creó *app/src/main/res/values/font_certs.xml* con los certificados de desarrollo y producción de Google Play Services, copiados del archivo de pruebas oficial del repositorio de AndroidX (The Android Open Source Project, 2022).

Fase 2. Tipografía Poppins

En *Type.kt* se definió el proveedor, la *GoogleFont("Poppins")* y una *FontFamily* con los pesos Normal, Medium, SemiBold y Bold. A diferencia de la guía, que solo sobrescribe *titleLarge* y *bodyLarge*, se aplicó Poppins a toda la jerarquía de *Typography* copiando los estilos por defecto de Material 3 con *.copy(fontFamily = ...)*; así, los campos del formulario, las etiquetas y los títulos de EcoTrip también cambian de fuente y la aplicación no mezcla dos familias. *titleLarge* (Bold, 22 sp) y *bodyLarge* (Normal, 16 sp) conservan exactamente los valores de la guía.

Fase 3. Activación del color dinámico

Theme.kt ya contenía la lógica pedida y se verificó sin modificarla: el parámetro *dynamicColor* vale *true* por defecto y la selección del esquema usa *dynamicDarkColorScheme* o *dynamicLightColorScheme* cuando *Build.VERSION.SDK_INT >= Build.VERSION_CODES.S*. En dispositivos anteriores a Android 12 se cae a *LightColorScheme* o *DarkColorScheme*, definidos con la paleta

estática de *Color.kt*. El resultado se transmite a *MaterialTheme* junto con la nueva tipografía.

Fase 4. Pantalla ConfigScreen con TopAppBar

Se agregó *ConfigScreen* en *MainActivity.kt*. Su *TopAppBar* usa *primaryContainer* como fondo y *onPrimaryContainer* para el título, según la guía. Además de la frase de bienvenida, la pantalla muestra cinco bloques que pintan los roles *primary*, *primaryContainer*, *secondaryContainer*, *tertiaryContainer* y *surfaceVariant* con su color de contenido, para que el cambio de paleta sea visible de un vistazo. Para poder abrirla desde la aplicación se creó la ruta tipada *ConfigRoute* y se añadió un ícono de ajustes en la barra del formulario; el parámetro es opcional, por lo que las pruebas de resiliencia existentes no se ven afectadas.

Desafío extra. FloatingActionButton

Dentro del *Scaffold* de *ConfigScreen* se añadió un *FloatingActionButton* con *containerColor = secondaryContainer*. Se agregó también *contentColor = onSecondaryContainer*, porque al cambiar solo el fondo el ícono mantendría el color de contenido del rol anterior y podría perder contraste.

Resultados

Las pruebas se hicieron en el emulador Pixel 8 con Android 16 (API 36). Se generaron dos imágenes de fondo de pantalla, una degradada en naranja y otra en azul, y se aplicaron con la acción «Establecer como fondo» del sistema. Tras cada cambio se reinició la aplicación para leer el nuevo esquema.



Figura 1. Fondos de pantalla de prueba aplicados al emulador: naranja (izquierda) y azul (derecha).

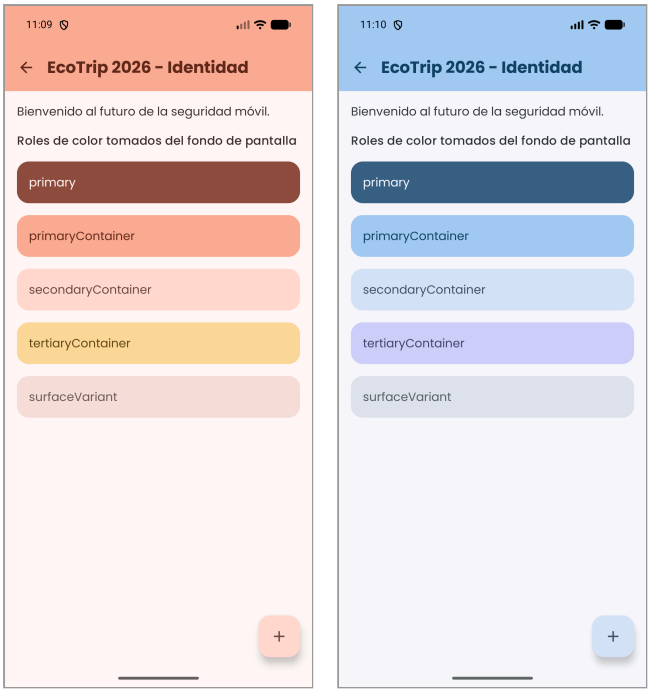


Figura 2. ConfigScreen con color dinámico: fondo naranja (izquierda) y fondo azul (derecha). Cambian la barra, los bloques de color y el botón flotante; la tipografía Poppins se mantiene.

La Figura 2 confirma el segundo resultado esperado de la guía: el *TopAppBar*, el *FloatingActionButton* y los textos cambian de tono sin tocar una sola línea de código. Con el fondo naranja la paleta es salmón y ámbar; con el azul, celeste y lavanda. La tipografía, en cambio, no depende del fondo: en ambas capturas los títulos se ven en Poppins Bold, con terminaciones redondeadas, que es el primer resultado esperado.

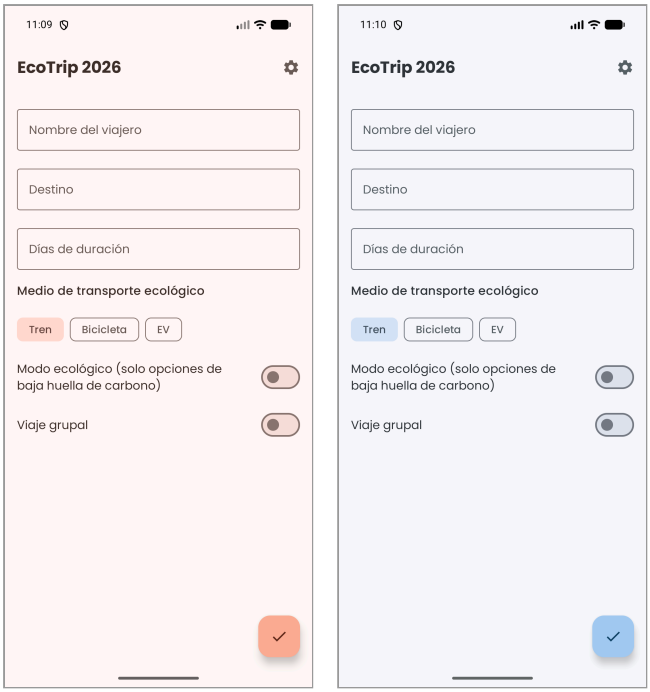


Figura 3. El formulario de EcoTrip, sin modificar su código de pantalla, adopta el color dinámico y Poppins: fondo naranja (izquierda) y fondo azul (derecha).

La Figura 3 muestra un beneficio adicional: como el tema es global, todas las pantallas previas de EcoTrip heredan la identidad visual (campos, *chips*, interruptores y botón de guardar) sin cambios individuales.

Influencia del colorScheme en la accesibilidad

El *colorScheme* es la capa donde se decide el contraste, y el contraste es la condición más básica de legibilidad para personas con baja visión, daltonismo o simplemente con la pantalla bajo el sol. El color dinámico introduce una incertidumbre nueva: el desarrollador ya no conoce de antemano los colores concretos. Material resuelve esto con roles emparejados cuyos tonos se calculan para garantizar una diferencia de luminosidad mínima (Google, s. f.-b), pero la garantía solo se cumple si el código respeta los pares. Poner texto *onSurface* sobre *primaryContainer*, o un color fijo como el negro sobre un fondo dinámico, rompe la garantía.

Para comprobarlo se midió el contraste real sobre las capturas de la Figura 2: se tomó el color del fondo de cada bloque y el píxel de texto más alejado de él, y se aplicó la fórmula de luminancia relativa de WCAG 2.2 (W3C, 2023). Como las capturas tienen suavizado de bordes, los valores son aproximaciones por defecto del contraste real.

Tabla 1

Contraste medido (relación de luminancia) entre cada rol y su color de contenido

Rol (fondo / contenido)	Fondo naranja	Fondo azul	Mínimo AA (texto normal)	Resultado
primaryContainer / onPrimaryContainer (TopAppBar)	6,12 : 1	6,05 : 1	4,5 : 1	Cumple
primary / onPrimary	6,02 : 1	6,06 : 1	4,5 : 1	Cumple
secondaryContainer / onSecondaryContainer	6,14 : 1	6,10 : 1	4,5 : 1	Cumple
tertiaryContainer / onTertiaryContainer	6,12 : 1	6,05 : 1	4,5 : 1	Cumple
surfaceVariant / onSurfaceVariant	4,95 : 1	4,94 : 1	4,5 : 1	Cumple

Los cinco pares superan 4,5:1 con ambos fondos, y los valores son casi idénticos entre una paleta y otra (alrededor de 6:1), lo que ilustra la regularidad tonal del sistema. El margen más estrecho lo tiene *surfaceVariant*, el rol que se usa para texto secundario: por eso conviene no atenuarlo más con transparencia. Estos resultados se refieren al modo claro; el modo oscuro usa tonos distintos y debe verificarse por separado.

Otros dos aspectos del *colorScheme* afectan a la accesibilidad. Primero, el significado no debe depender solo del color: en la aplicación el estado «Activado/Desactivado» del resumen se expresa con texto, no únicamente con un tono. Segundo, la tipografía también cuenta: Poppins tiene aperturas amplias y buena altura de x, pero su peso Bold en 22 sp para títulos mantiene la jerarquía sin depender del color, y el cuerpo de 16 sp respeta el tamaño mínimo recomendado.

Dificultades encontradas

Tres situaciones merecen registro. (1) El proyecto no tenía *font_certs.xml*; sin él, *Type.kt* no compila porque *R.array.com_google_android_gms_fonts_certs* no existe. (2) La propiedad *org.gradle.java.home* del repositorio apunta a una ruta del equipo de otro integrante; en este equipo se compiló sustituyéndola por línea de comandos (JDK 21) sin modificar el archivo compartido. (3) La primera vez que se carga una fuente descargable el emulador necesita conexión a Internet y Google Play Services; la guía lo menciona como requisito previo y fue efectivamente necesario.

Conclusiones

El color dinámico logra que una aplicación se sienta parte del dispositivo con muy poco código: bastó con verificar una bandera y un bloque *when* en *Theme.kt*, y todas las pantallas de EcoTrip cambiaron de paleta a la vez. Lo que en realidad hace el trabajo es la disciplina de usar los roles del *colorScheme* en lugar de colores fijos, y esa disciplina es la que sostiene la accesibilidad: con cinco pares de roles y dos fondos muy distintos, el contraste se mantuvo entre 4,9:1 y 6,1:1, por encima del mínimo AA.

La tipografía aporta identidad con poco costo, pero introduce una dependencia de red y de Google Play Services. Mi valoración es que conviene aceptarla en aplicaciones con servicios de Google, y declarar una fuente de respaldo (la *FontFamily* por defecto) para dispositivos sin ellos. Además, aplicar Poppins solo a dos estilos, como sugiere la guía, habría dejado la aplicación con dos familias mezcladas; extenderla a toda la jerarquía fue una decisión que mejoró el resultado visual.

Por último, el laboratorio muestra que el diseño y la ingeniería no compiten: el mismo cambio que hizo la interfaz más moderna fue el que habría roto la legibilidad si se hubiera hecho sin respetar los pares de color. Medir el contraste, aun de forma aproximada, convirtió una afirmación de la teoría en un dato verificable.

Recomendaciones

1. Usar siempre pares de roles (*x* sobre *onX*) y evitar colores literales dentro de componentes; reservarlos para la paleta de respaldo.

2. Probar con al menos dos fondos muy distintos (uno claro y saturado, otro oscuro) y en modo claro y oscuro antes de dar por buena una pantalla.

3. Declarar una fuente de respaldo para dispositivos sin Google Play Services, por ejemplo con *FontFamily(Font(googleFont = ...), Font(DeviceFontFamilyName("sans-serif")))*.

4. Automatizar la comprobación de contraste con el Escáner de Accesibilidad de Android o con pruebas de interfaz que inspeccionen el árbol semántico.

5. No fijar *dynamicColor = true* como única opción: ofrecer un ajuste para desactivarlo permite a una marca con colores obligatorios (por ejemplo, el verde de EcoTrip) conservar su identidad.

6. Mover el ajuste de color dinámico y el acceso a la pantalla de identidad a una pantalla de configuración definitiva antes de publicar la aplicación.

Referencias

Android Developers. (s. f.-a). *Material Design 3 in Compose*. Google. https://developer.android.com/develop/ui/compose/designsystems/material3

Android Developers. (s. f.-b). *Fonts in Compose: Use downloadable fonts*. Google. https://developer.android.com/develop/ui/compose/text/fonts

Android Developers. (s. f.-c). *Accessibility in Compose*. Google. https://developer.android.com/develop/ui/compose/accessibility

Google. (s. f.-a). *Dynamic color*. Material Design 3. https://m3.material.io/styles/color/dynamic-color/overview

Google. (s. f.-b). *Color roles*. Material Design 3. https://m3.material.io/styles/color/roles

The Android Open Source Project. (2022). *font_certs.xml* [Archivo de código]. AndroidX, compose/ui/ui-text-google-fonts. https://github.com/androidx/androidx

World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/

Anexo. Código modificado

app/src/main/java/com/example/myapplication/MainActivity.kt

```
package com.example.myapplication

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material.icons.filled.Add
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.FloatingActionButton
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.material3.TopAppBarDefaults
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import com.example.myapplication.navigation.EcoTripNavHost
import com.example.myapplication.ui.theme.MyApplicationTheme
import com.example.myapplication.ui.viewmodel.EcoTripViewModel

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MyApplicationTheme {
                val ecoTripViewModel: EcoTripViewModel = viewModel(
                    factory = EcoTripViewModel.factory(application)
                )
                EcoTripNavHost(viewModel = ecoTripViewModel)
            }
        }
    }
}

/** Pantalla del Laboratorio 7.2: muestra cómo reacciona la interfaz al Color Dinámico y a Poppins. */
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ConfigScreen(onVolver: (() -> Unit)? = null) {
    Scaffold(
        topBar = {
            TopAppBar(
                title = {
                    Text("EcoTrip 2026 - Identidad", style = MaterialTheme.typography.titleLarge)
                },
                navigationIcon = {
                    if (onVolver != null) {
                        IconButton(onClick = onVolver) {
                            Icon(
                                imageVector = Icons.AutoMirrored.Filled.ArrowBack,
```

```
                                contentDescription = "Volver al formulario"
                            )
                        }
                    }
                },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = MaterialTheme.colorScheme.primaryContainer,
                    titleContentColor = MaterialTheme.colorScheme.onPrimaryContainer,
                    navigationIconContentColor = MaterialTheme.colorScheme.onPrimaryContainer
                )
            )
        },
        floatingActionButton = {
            FloatingActionButton(
                onClick = { /* Acción */ },
                containerColor = MaterialTheme.colorScheme.secondaryContainer,
                contentColor = MaterialTheme.colorScheme.onSecondaryContainer
            ) {
                Icon(Icons.Default.Add, contentDescription = "Agregar")
            }
        }
    ) { innerPadding ->
        Column(
            modifier = Modifier
                .padding(innerPadding)
                .verticalScroll(rememberScrollState())
                .padding(16.dp),
            verticalArrangement = Arrangement.spacedBy(16.dp)
        ) {
            Text(
                text = "Bienvenido al futuro de la seguridad móvil.",
                style = MaterialTheme.typography.bodyLarge
            )
            Text(
                text = "Roles de color tomados del fondo de pantalla",
                style = MaterialTheme.typography.titleMedium
            )
            val colores = MaterialTheme.colorScheme
            ColorRole("primary", colores.primary, colores.onPrimary)
            ColorRole("primaryContainer", colores.primaryContainer, colores.onPrimaryContainer)
            ColorRole("secondaryContainer", colores.secondaryContainer, colores.onSecondaryContainer)
            ColorRole("tertiaryContainer", colores.tertiaryContainer, colores.onTertiaryContainer)
            ColorRole("surfaceVariant", colores.surfaceVariant, colores.onSurfaceVariant)
        }
    }
}

@Composable
private fun ColorRole(nombre: String, fondo: Color, contenido: Color) {
    Box(
        modifier = Modifier
            .fillMaxWidth()
            .height(56.dp)
            .background(fondo, RoundedCornerShape(16.dp))
            .padding(horizontal = 16.dp),
        contentAlignment = Alignment.CenterStart
    ) {
        Text(text = nombre, color = contenido, style = MaterialTheme.typography.bodyLarge)
    }
}
```

app/src/main/java/com/example/myapplication/ui/theme/Theme.kt

```
package com.example.myapplication.ui.theme

import android.app.Activity
import android.os.Build
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.darkColorScheme
import androidx.compose.material3.dynamicDarkColorScheme
import androidx.compose.material3.dynamicLightColorScheme
import androidx.compose.material3.lightColorScheme
```

```
import androidx.compose.runtime.Composable
import androidx.compose.ui.platform.LocalContext

private val DarkColorScheme = darkColorScheme(
    primary = Purple80,
    secondary = PurpleGrey80,
    tertiary = Pink80
)

private val LightColorScheme = lightColorScheme(
    primary = Purple40,
    secondary = PurpleGrey40,
    tertiary = Pink40

    /* Other default colors to override
    background = Color(0xFFFFFBFE),
    surface = Color(0xFFFFFBFE),
    onPrimary = Color.White,
    onSecondary = Color.White,
    onTertiary = Color.White,
    onBackground = Color(0xFF1C1B1F),
    onSurface = Color(0xFF1C1B1F),
    */
)

@Composable
fun MyApplicationTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    // Dynamic color is available on Android 12+
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context) else dynamicLightColorScheme(context)
        }

        darkTheme -> DarkColorScheme
        else -> LightColorScheme
    }

    MaterialTheme(
        colorScheme = colorScheme,
        typography = Typography,
        content = content
    )
}
```

app/src/main/java/com/example/myapplication/ui/theme/Type.kt

```
package com.example.myapplication.ui.theme

import androidx.compose.material3.Typography
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.googlefonts.Font
import androidx.compose.ui.text.googlefonts.GoogleFont
import androidx.compose.ui.unit.sp
import com.example.myapplication.R

// Proveedor de fuentes: Google Play Services descarga y cachea la fuente bajo demanda.
val provider = GoogleFont.Provider(
    providerAuthority = "com.google.android.gms.fonts",
    providerPackage = "com.google.android.gms",
    certificates = R.array.com_google_android_gms_fonts_certs
)

val fontName = GoogleFont("Poppins")

val poppinsFontFamily = FontFamily(
    Font(googleFont = fontName, fontProvider = provider, weight = FontWeight.Normal),
```

```
    Font(googleFont = fontName, fontProvider = provider, weight = FontWeight.Medium),
    Font(googleFont = fontName, fontProvider = provider, weight = FontWeight.SemiBold),
    Font(googleFont = fontName, fontProvider = provider, weight = FontWeight.Bold)
)

private val base = Typography()

// Poppins en toda la jerarquía de Material 3; titleLarge y bodyLarge fijan los valores del laboratorio.
val Typography = Typography(
    displayLarge = base.displayLarge.copy(fontFamily = poppinsFontFamily),
    displayMedium = base.displayMedium.copy(fontFamily = poppinsFontFamily),
    displaySmall = base.displaySmall.copy(fontFamily = poppinsFontFamily),
    headlineLarge = base.headlineLarge.copy(fontFamily = poppinsFontFamily),
    headlineMedium = base.headlineMedium.copy(fontFamily = poppinsFontFamily),
    headlineSmall = base.headlineSmall.copy(fontFamily = poppinsFontFamily),
    titleLarge = TextStyle(
        fontFamily = poppinsFontFamily,
        fontWeight = FontWeight.Bold,
        fontSize = 22.sp
    ),
    titleMedium = base.titleMedium.copy(fontFamily = poppinsFontFamily),
    titleSmall = base.titleSmall.copy(fontFamily = poppinsFontFamily),
    bodyLarge = TextStyle(
        fontFamily = poppinsFontFamily,
        fontWeight = FontWeight.Normal,
        fontSize = 16.sp
    ),
    bodyMedium = base.bodyMedium.copy(fontFamily = poppinsFontFamily),
    bodySmall = base.bodySmall.copy(fontFamily = poppinsFontFamily),
    labelLarge = base.labelLarge.copy(fontFamily = poppinsFontFamily),
    labelMedium = base.labelMedium.copy(fontFamily = poppinsFontFamily),
    labelSmall = base.labelSmall.copy(fontFamily = poppinsFontFamily)
)
```

app/src/main/res/values/font_certs.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!--
  Copyright 2022 The Android Open Source Project

  Licensed under the Apache License, Version 2.0 (the "License");
  you may not use this file except in compliance with the License.
  You may obtain a copy of the License at

       http://www.apache.org/licenses/LICENSE-2.0

  Unless required by applicable law or agreed to in writing, software
  distributed under the License is distributed on an "AS IS" BASIS,
  WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
  See the License for the specific language governing permissions and
  limitations under the License.
  -->
<resources>
    <array name="com_google_android_gms_fonts_certs">
        <item>@array/com_google_android_gms_fonts_certs_dev</item>
        <item>@array/com_google_android_gms_fonts_certs_prod</item>
    </array>
    <string-array name="com_google_android_gms_fonts_certs_dev">
        <item>

MIIEqDCCA5CgAwIBAgIJANWFuGx90071MA0GCSqGSIb3DQEBBAUAMIGUMQswCQYDVQQGEwJVUzETMBEGA1UECBMKQ2FsaWZvcm5pYTEW
MBQGA1UEBxMNTW91bnRhaW4gVmlldzEQMA4GA1UEChMHQW5kcm9pZDEQMA4GA1UECxMHQW5kcm9pZDEQMA4GA1UEAxMHQW5kcm9pZDEi
MCAGCSqGSIb3DQEJARYTYW5kcm9pZEBhbmRyb2lkLmNvbTAeFw0wODA0MTUyMzM2NTZaFw0zNTA5MDEyMzM2NTZaMIGUMQswCQYDVQQG
EwJVUzETMBEGA1UECBMKQ2FsaWZvcm5pYTEWMBQGA1UEBxMNTW91bnRhaW4gVmlldzEQMA4GA1UEChMHQW5kcm9pZDEQMA4GA1UECxMH
QW5kcm9pZDEQMA4GA1UEAxMHQW5kcm9pZDEiMCAGCSqGSIb3DQEJARYTYW5kcm9pZEBhbmRyb2lkLmNvbTCCASAwDQYJKoZIhvcNAQEB
BQADggENADCCAQgCggEBANbOLggKv+IxTdGNs8/TGFy0PTP6DHThvbbR24kT9ixcOd9W+EaBPWW+wPPKQmsHxajtWjmQwWfna8mZuSeJ
S48LIgAZlKkpFeVyxW0qMBujb8X8ETrWy550NaFtI6t9+u7hZeTfHwqNvacKhp1RbE6dBRGWynwMVX8XW8N1+UjFaq6GCJukT4qmpN2a
fb8sCjUigq0GuMwYXrFVee74bQgLHWGJwPmvmLHC69EH6kWr22ijx4OKXlSIx2xT1AsSHee70w5iDBiK4aph27yH3TxkXy9V89TDdexA
cKk/cVHYNnDBapcavl7y0RiQ4biu8ymM8Ga/nmzhRKya6G0cGw8CAQOjgfwwgfkwHQYDVR0OBBYEFI0cxb6VTEM8YYY6FbBMvAPyT+Cy
MIHJBgNVHSMEgcEwgb6AFI0cxb6VTEM8YYY6FbBMvAPyT+CyoYGapIGXMIGUMQswCQYDVQQGEwJVUzETMBEGA1UECBMKQ2FsaWZvcm5p
YTEWMBQGA1UEBxMNTW91bnRhaW4gVmlldzEQMA4GA1UEChMHQW5kcm9pZDEQMA4GA1UECxMHQW5kcm9pZDEQMA4GA1UEAxMHQW5kcm9p
ZDEiMCAGCSqGSIb3DQEJARYTYW5kcm9pZEBhbmRyb2lkLmNvbYIJANWFuGx90071MAwGA1UdEwQFMAMBAf8wDQYJKoZIhvcNAQEEBQAD
ggEBABnTDPEF+3iSP0wNfdIjIz1AlnrPzgAIHVvXxunW7SBrDhEglQZBbKJEk5kT0mtKoOD1JMrSu1xuTKEBahWRbqHsXclaXjoBADb0
```

```
kkjVEJu/Lh5hgYZnOjvlba8Ld7HCKePCVePoTJBdI4fvugnL8TsgK05aIskyY0hKI9L8KfqfGTl1lzOv2KoWD0KWwtAWPoGChZxmQ+nB
li+gwYMzM1vAkP+aayLe0a1EQimlOalO762r0GXO0ks+UeXde2Z4e+8S/pf7pITEI/tP+MxJTALw9QUWEv9lKTk+jkbqxbsh8nfBUapf
KqYn0eidpwq2AzVp3juYl7//fKnaPhJD9gs=
        </item>
    </string-array>
    <string-array name="com_google_android_gms_fonts_certs_prod">
        <item>

MIIEQzCCAyugAwIBAgIJAMLgh0ZkSjCNMA0GCSqGSIb3DQEBBAUAMHQxCzAJBgNVBAYTAlVTMRMwEQYDVQQIEwpDYWxpZm9ybmlhMRYw
FAYDVQQHEw1Nb3VudGFpbiBWaWV3MRQwEgYDVQQKEwtHb29nbGUgSW5jLjEQMA4GA1UECxMHQW5kcm9pZDEQMA4GA1UEAxMHQW5kcm9p
ZDAeFw0wODA4MjEyMzEzMzRaFw0zNjAxMDcyMzEzMzRaMHQxCzAJBgNVBAYTAlVTMRMwEQYDVQQIEwpDYWxpZm9ybmlhMRYwFAYDVQQH
Ew1Nb3VudGFpbiBWaWV3MRQwEgYDVQQKEwtHb29nbGUgSW5jLjEQMA4GA1UECxMHQW5kcm9pZDEQMA4GA1UEAxMHQW5kcm9pZDCCASAw
DQYJKoZIhvcNAQEBBQADggENADCCAQgCggEBAKtWLgDYO6IIrgqWbxJOKdoR8qtW0I9Y4sypEwPpt1TTcvZApxsdyxMJZ2JORland2qS
GT2y5b+3JKkedxiLDmpHpDsz2WCbdxgxRczfey5YZnTJ4VZbH0xqWVW/8lGmPav5xVwnIiJS6HXk+BVKZF+JcWjAsb/GEuq/eFdpuzSq
eYTcfi6idkyugwfYwXFU1+5fZKUaRKYCwkkFQVfcAs1fXA5V+++FGfvjJ/CxURaSxaBvGdGDhfXE28LWuT9ozCl5xw4Yq5OGazvV24mZ
VSoOO0yZ31j7kYvtwYK6NeADwbSxDdJEqO4k//0zOHKrUiGYXtqw/A0LFFtqoZKFjnkCAQOjgdkwgdYwHQYDVR0OBBYEFMd9jMIhF1Yl
mn/Tgt9r45jk14alMIGmBgNVHSMEgZ4wgZuAFMd9jMIhF1Ylmn/Tgt9r45jk14aloXikdjB0MQswCQYDVQQGEwJVUzETMBEGA1UECBMK
Q2FsaWZvcm5pYTEWMBQGA1UEBxMNTW91bnRhaW4gVmlldzEUMBIGA1UEChMLR29vZ2xlIEluYy4xEDAOBgNVBAsTB0FuZHJvaWQxEDAO
BgNVBAMTB0FuZHJvaWSCCQDC4IdGZEowjTAMBgNVHRMEBTADAQH/MA0GCSqGSIb3DQEBBAUAA4IBAQBt0lLO74UwLDYKqs6Tm8/yzKkE
u116FmH4rkaymUIE0P9KaMftGlMexFlaYjzmB2OxZyl6euNXEsQH8gjwyxCUKRJNexBiGcCEyj6z+a1fuHHvkiaai+KL8W1EyNmgjmyy
8AW7P+LLlkR+ho5zEHatRbM/YAnqGcFh5iZBqpknHf1SKMXFh4dd239FJ1jWYfbMDMy3NS5CTMQ2XFI1MvcyUTdZPErjQfTbQe3aDQsQ
cafEQPD+nqActifKZ0Np0IS9L9kR/wbNvyz6ENwPiTrjV2KRkEjH78ZMcUQXg0L3BYHJ3lc69Vs5Ddf9uUGGMYldX3WfMBEmh/9iFBDA
aTCK
        </item>
    </string-array>
</resources>
```

app/build.gradle.kts

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.kotlin.compose)
    alias(libs.plugins.kotlin.serialization)
}

android {
    namespace = "com.example.myapplication"
    compileSdk = 36

    defaultConfig {
        applicationId = "com.example.myapplication"
        minSdk = 26
        targetSdk = 36
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            isMinifyEnabled = false
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_21
        targetCompatibility = JavaVersion.VERSION_21
    }
    kotlinOptions {
        jvmTarget = "21"
    }
    buildFeatures {
        compose = true
    }
}

dependencies {
    implementation(libs.androidx.core.ktx)
```

```
    implementation(libs.androidx.lifecycle.runtime.ktx)
    implementation(libs.androidx.activity.compose)
    implementation(platform(libs.androidx.compose.bom))
    implementation(libs.androidx.ui)
    implementation(libs.androidx.ui.graphics)
    implementation(libs.androidx.ui.tooling.preview)
    implementation(libs.androidx.material3)
    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.junit)
    androidTestImplementation(libs.androidx.espresso.core)
    androidTestImplementation(platform(libs.androidx.compose.bom))
    androidTestImplementation(libs.androidx.ui.test.junit4)
    debugImplementation(libs.androidx.ui.tooling)
    debugImplementation(libs.androidx.ui.test.manifest)
    implementation(libs.androidx.lifecycle.viewmodel.compose)
    implementation(libs.androidx.navigation.compose)
    implementation(libs.kotlinx.serialization.json)
    implementation(libs.androidx.datastore.preferences)
    implementation(libs.androidx.ui.text.google.fonts)
}
```

gradle/libs.versions.toml

```
[versions]
agp = "8.7.0"
kotlin = "2.0.0"
coreKtx = "1.13.1"
junit = "4.13.2"
junitVersion = "1.2.1"
espressoCore = "3.7.0"
lifecycleRuntimeKtx = "2.8.6"
activityCompose = "1.9.3"
composeBom = "2024.09.03"
# Nuevas versiones requeridas para el Examen RDA-2
navigationCompose = "2.8.3"
kotlinxSerialization = "1.6.3"
datastorePreferences = "1.1.1"

[libraries]
androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
junit = { group = "junit", name = "junit", version.ref = "junit" }
androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref =
"espressoCore" }
androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx",
version.ref = "lifecycleRuntimeKtx" }
androidx-lifecycle-viewmodel-compose = { group = "androidx.lifecycle", name = "lifecycle-viewmodel-
compose", version.ref = "lifecycleRuntimeKtx" }
androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref =
"activityCompose" }
androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeBom" }
androidx-ui = { group = "androidx.compose.ui", name = "ui" }
androidx-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }
androidx-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }
androidx-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }
androidx-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }
androidx-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }
androidx-ui-text-google-fonts = { group = "androidx.compose.ui", name = "ui-text-google-fonts", version
= "1.7.0" }
androidx-material3 = { group = "androidx.compose.material3", name = "material3" }

# Rutas Tipadas y Serialización
androidx-navigation-compose = { group = "androidx.navigation", name = "navigation-compose", version.ref
= "navigationCompose" }
kotlinx-serialization-json = { group = "org.jetbrains.kotlinx", name = "kotlinx-serialization-json",
version.ref = "kotlinxSerialization" }

# Persistencia en Disco
androidx-datastore-preferences = { group = "androidx.datastore", name = "datastore-preferences",
version.ref = "datastorePreferences" }

[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
```

```
kotlin-android = { id = "org.jetbrains.kotlin.android", version.ref = "kotlin" }
kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }
kotlin-serialization = { id = "org.jetbrains.kotlin.plugin.serialization", version.ref = "kotlin" }
```